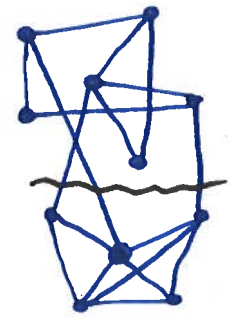
Lecture 15 The Spectral Method for Graph Partitioning

Graph partitioning is a classic problem in computer science, studied since the 1960s.

Problem: [Divide the vertices of a graph into a given number of non-overlapping groups of given sizes such that the number of edges between groups is minimized.]

Example



$R = 3$
$n = 11$
$k = 2$
$n_1 = 6, n_2 = 5$

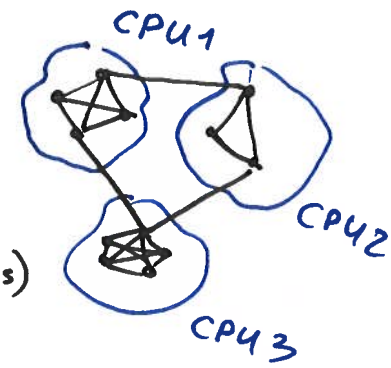
This problem arises in a variety of contexts, but most prominently in the development of computer algorithms for parallel or distributed computation.

Suppose a computation consists of n tasks, each to be carried out by a separate program on one of k CPUs. Typically there is a desired number of tasks (volume of work) to be assigned to each of the CPUs, $n_1 + n_2 + \ldots + n_k = n$. If the CPUs are identical and the tasks are of similar complexity $\Rightarrow n_1 = n_2 = \ldots = n_k = n/k$. It is also usually the case that the individual tasks must communicate with one another to complete the overall computation. We can represent the pattern of communication as a graph with n vertices representing the tasks and edges connecting pairs of tasks that need to communicate.

Remark In practice, the amount of communication between different pairs of tasks could vary, and this leads to a weighted graph.

Communications between CPUs is slow compared to data movement within CPU. Therefore, we want to keep such communication to a minimum.

This leads to the graph partitioning problem:



Divide n vertices (tasks) into k groups (CPUs) with n_i vertices in group i, $n_1 + \ldots + n_k = n$, such that the number of edges between groups is minimized.

cut size

Let's focus on the graph bisection problem, a special case with $k=2$

Remark: It is not really a big limitation since in practice graph partitioning is often solved by a sequence of bisection problems.

Why is partitioning is hard?

The most straightforward way to solve the problem is so-called exhaustive search: look at all possible partitions of the required sizes and choose the one with the smallest cut size.

Let $N(n_1, n_2)$ be the number of partitions of a graph of n vertices into two groups of n_1 and n_2 vertices. Then

$$N(n_1,n_2) = \binom{n}{n_1} = \frac{n!}{n_1!\,(n-n_1)!} = \frac{n!}{n_1!\,n_2!} \approx \frac{\sqrt{2\pi n}\,(n/e)^n}{\sqrt{2\pi n_1}\,(n_1/e)^{n_1}\sqrt{2\pi n_2}\,(n_2/e)^{n_2}}$$

Stirling's formula $n! \approx \underbrace{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n}_{= \ast\cdot(1+O(\frac{1}{n}))}$

$$= \frac{n^{n+\frac{1}{2}}}{n_1^{n_1+\frac{1}{2}} \cdot n_2^{n_2+\frac{1}{2}}\sqrt{2\pi}}$$

For example, if $n_1 = n_2 = n/2 \Rightarrow N\left(\frac{n}{2},\frac{n}{2}\right) = \dfrac{n^{n+\frac{1}{2}}}{\left(\left(\frac{n}{2}\right)^{\frac{n+1}{2}}\right)^2\sqrt{2\pi}} = \dfrac{n^{n+\frac{1}{2}}\cdot 2^{n+1}}{n^{n+1}\sqrt{2\pi}}$

And so $\boxed{N\left(\frac{n}{2},\frac{n}{2}\right) = \frac{2^{n+1}}{\sqrt{n2\pi}}}$ ← grows roughly exponentially with the size of the graph.

Thus, exhaustive search is computationally prohibitive.

Remark: Values up to about $n=30$ are feasible with current computers, but not much beyond that.

Certain fundamental results in computer science tell us that either an algorith can be clever and run quickly, but will fail to find the optimal solution in some (perhaps most) cases, or it always finds the optimal solution but takes an impractical length of time to do it.

The good news is that there are heuristic algorithms that find approximate, but acceptable for many applications, solution in a reasonable time.

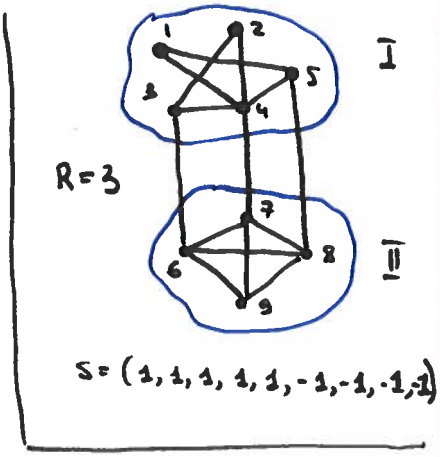
Spectral Partitioning

The spectral partitioning method developed by Miroslav Fiedler in 1973 is one of the most widely used heuristic partitioning methods.

Consider a graph consisting of n vertices and a division of graph vertices into two groups with n_1 vertices in group 1 and n_2 vertices in group 2.

Let A be the adjacency matrix for the graph

$$A_{ij} = \begin{cases} 1 & \text{if vertices } i \text{ and } j \text{ are connected,} \\ 0 & \text{otherwise} \end{cases}$$



Then the cut size is $R = \frac{1}{2} \sum_{\substack{i,j \text{ in} \\ \text{diff. groups}}} A_{ij}$

Let's define an index vector $s \in \mathbb{R}^n$ which fully describes the graph division:

$$s_i = \begin{cases} +1 & \text{if } i \text{ belongs to group 1,} \\ -1 & \text{if } i \text{ belongs to group 2.} \end{cases}$$

hyper cube

$s \in$ $\subset \mathbb{R}^n$

$\leftarrow \sum s_i = n_1 - n_2$

We want $R = \frac{1}{2} \sum_{i,j} x_{ij} A_{ij}$, where $x_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are in diff. groups} \\ 0 & \text{if } i \text{ and } j \text{ are in the same group.} \end{cases}$

Thus, $R = \frac{1}{4} \sum_{i,j} (1 - s_i s_j) A_{ij}$ $\qquad x_{ij} = \frac{1}{2}(1 - s_i s_j)$

Our next goal is to re-write this expression in a more compact matrix form.

$$R = \frac{1}{4}\Big(\sum_{i,j} A_{ij} - \underbrace{\sum_{i,j} s_i s_j A_{ij}}_{s^T A s} \Big) \underset{\text{we want}}{=} \frac{1}{4} \sum_{i,j} s_i s_j L_{ij} = \frac{1}{4} s^T L s$$

Let's play with the first term:

$$\sum_{i,j} A_{ij} = \sum_i \Big(\underbrace{\sum_j A_{ij}}_{\text{degree of } i} \Big) = \sum_i k_i = \sum_i k_i s_i^2 = \sum_{ij} k_i \delta_{ij} s_i s_j$$

Thus, $R = \frac{1}{4} \Big(\sum_{i,j} s_i s_j \underbrace{(k_i \delta_{ij} - A_{ij})}_{L_{ij}} \Big) = \frac{1}{4} s^T L s$

the Laplacian matrix of the graph

↓ appears in many contexts
- diffusion on graphs
- random walks on graphs

$$L_{ij} = k_i \delta_{ij} - A_{ij} = \begin{cases} k_i & \text{if } i = j \\ 0 & \text{if } i \neq j \text{ and not connected} \\ -1 & \text{if } i \neq j \text{ and connected} \end{cases}$$

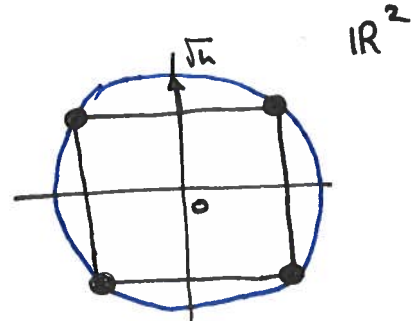
$$L = D - A = \begin{bmatrix} k_1 & & 0 \\ & \ddots & \\ 0 & & k_n \end{bmatrix}$$

L has interpret. of the discrete Laplace operator

So, a matrix formulation of the graph partitioning problem is the following discrete constrained optimization problem:

$$(\#)\quad \begin{cases} s^* = \arg\min\limits_{s\in\{-1,1\}^n} s^T L s \quad , \text{ where } L \text{ is the graph Laplacian} \\ \text{s.t. } \sum\limits_{i=1}^{n} s_i = n_1 - n_2 \end{cases}$$

This problem is very hard. The difficulty stems from the discreteness of $s_i = \pm 1$. If s_i were continuous, we could simply differentiate to find the minimum. This suggests an approximation approach (often used in many areas of applied math), called the relaxation method. The idea is to relax the constraint that makes our problem difficult.



Our vector $s \in \{-1,1\}^n$ is one of the 2^n corners of an n-dim hypercube centered at the origin of $\mathbb{R}^n$. The norm $\|s\| = \sqrt{n}$. Let's relax this constraint by allowing $s \in \mathbb{S}_{\sqrt{n}}$ to be anywhere on the sphere of radius $\sqrt{n}$. So we make our search space a bit larger: $\{-1,1\}^n \subset \mathbb{S}_{\sqrt{n}}$.

So, the partitioning problem in the relaxed form:

$$(\tilde{\#})\quad \begin{cases} s^* = \arg\min\limits_{s\in\mathbb{R}^n} s^T L s \\ \text{s.t. } \sum\limits_{i=1}^{n} s_i^2 = n \\ \sum\limits_{i=1}^{n} s_i = n_1 - n_2 \end{cases}$$

This problem is a standard piece of Calculus. It can be solved by the method of Lagrange multipliers. But there is a more elegant solution based of linear algebra and properties of the Laplacian.

Properties of the Laplacian are subject of hundreds of papers in math and phys. literature, and now are well understood. We will need a few of them.

① Laplacian is symmetric. $(L = D - A)$

② Its eigenvalues are non-negative (i.e. L is positive semi-definite)

If $\lambda \in \operatorname{spec} L \Rightarrow L v = \lambda v$

$\Rightarrow \lambda \|v\|^2 = v^T L v = v^T M^T M v = \|Mv\|^2$

$\Rightarrow \lambda \geq 0$

Let $i>j$, $e=\{i,j\}$: $M_{ev} = \begin{cases} 1 & v=i \\ -1 & v=j \\ 0 & \text{otherwise} \end{cases}$ $\quad M \in \mathbb{M}^{|E|\times|V|}$ incidence matrix $\Rightarrow L = M^T M$

③ $\lambda = 0$ is an eigenvalue of L with eigenvector $\mathbb{1} = (1, \dots, 1)^T$.

Indeed $L\mathbb{1} = (D-A)\mathbb{1} = \begin{bmatrix} k_1 \\ \vdots \\ k_n \end{bmatrix} - \begin{bmatrix} k_1 \\ \vdots \\ k_n \end{bmatrix} = \mathbf{0}$

④ Algebraic multiplicity of $\lambda = 0$ = # connected components of the graph.

strong assumption

not really ~~important (practical)~~

in practice, component are treated separatly.

Assume our graph is connected. Then $\lambda = 0$ is simple and

$$\operatorname{spec} L = \{\lambda_1 = 0 < \lambda_2 \le \dots \le \lambda_n\},$$

$$V_{\lambda_1 = 0} = \operatorname{span}(\mathbb{1}).$$

Since L is symmetric, it has an orthonormal eigen basis $\underset{\lambda_1=0}{\overset{\uparrow}{v_1 = \frac{\mathbb{1}}{\sqrt{n}}}}, \underset{\lambda_2}{\overset{\uparrow}{v_2}}, \dots, \underset{\lambda_n}{\overset{\uparrow}{v_n}}$.

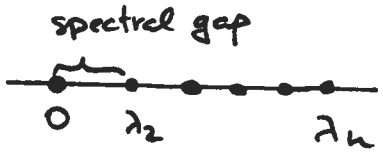
Let's express the vector s we are looking for in terms of this basis:

$$s = \sum_{i=1}^{n} \alpha_i v_i = \underbrace{\begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}}_{V} \underbrace{\begin{bmatrix} \\ \\ \end{bmatrix}}_{\alpha} = V \cdot \alpha$$

Then the problem $(\tilde{\#})$ can be re-written as follows:

- $s^T L s = \alpha^T V^T L V \alpha = \alpha^T V^T \boxed{\lambda_1 v_1, \dots, \lambda_n v_n} \alpha = \alpha^T \boxed{\begin{matrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{matrix}} \alpha = \sum_{i=2}^{n} \lambda_i \alpha_i^2$
- $\sum_{i=1}^{n} s_i^2 = s^T s = \alpha^T V^T V \alpha = \alpha^T \alpha = \sum_{i=1}^{n} \alpha_i^2$
- $\sum_{i=1}^{n} s_i = \mathbb{1}^T s = \mathbb{1}^T V \alpha \overset{\text{since } \mathbb{1} \text{ is orthogonal to } v_2 \dots v_n}{=} [\sqrt{n}, 0, \dots, 0]\, \alpha = \sqrt{n} \cdot \alpha_1$

$$\begin{cases} \alpha^* = \arg\min\limits_{\alpha \in \mathbb{R}^n} (\lambda_2 \alpha_2^2 + \dots + \lambda_n \alpha_n^2) \\ \text{s.t.} \quad \left.\begin{aligned} &\alpha_1^2 + \dots + \alpha_n^2 = n \\ &\alpha_1 = \frac{n_1 - n_2}{\sqrt{n}} \end{aligned}\right\} \iff \alpha_2^2 + \dots + \alpha_n^2 = n - \frac{(n_1 - n_2)^2}{n} = \frac{4 n_1 n_2}{n} \end{cases}$$



Solution is obvious: $\alpha_1 = \frac{n_1 - n_2}{\sqrt{n}}$, $\alpha_2 = \pm 2\sqrt{\frac{n_1 n_2}{n}}$, $\alpha_3 = \dots = \alpha_4 = 0$

In terms of s^*:

$$\boxed{s^* = \frac{n_1 - n_2}{n} \mathbb{1} + \alpha_2 v_2}$$

v_2 is an normalized $\|v_2\| = 1$ eigenvector of L that corresponds to λ_2, the smallest non-zero eigenvalue.

λ_2 is called the spectral gap or algebraic connectivity.

Fiedler value, v_2 is the Fiedler vector

The corresponding cut size is

$$R^* = \frac{1}{4} s^{*T} L s^* = \frac{1}{4}\left(\alpha_1 \mathbb{1}^T + \alpha_2 v_2^T\right) \overbrace{L\left(\alpha_1 \mathbb{1} + \alpha_2 v_2\right)}^{\alpha_2 \lambda_2 v_2} = \frac{\lambda_2 \alpha_2^2}{4} = \frac{n_1 n_2}{n} \lambda_2$$

So $R^* \propto \lambda_2 \Rightarrow \lambda_2$ is a measure of the graph connectivity.

if $\lambda_2 \ll 1 \Rightarrow$ there are good cuts.

Are we done? Of course not ☺ Recall that $s_i = \pm 1$ which is most likely not the case for $s^* = \alpha_1 \mathbb{1} + \alpha_2 v_2$ (this is continuous solution but we need discrete).

There are two "symmetric" natural ways to cook up a discrete (± 1) s from s^* (since $\alpha_2 = \pm$) (with n_1 "+1" and n_2 "−1") (most positive): (we in fact maximize $(s^T \cdot s^*) \to \max$) to make s as similar to s^* as possible

(a) Take n_1 largest s_i^* and replace them by "+1". Replace the remaining s_i^* by "−1".] s_+

(b) Take n_1 smallest (most negative) s_i^* and replace them by "+1" Replace n_2 remaining s_i^* by "−1".] s_-

Finally, we can compute the cut size for both s_+ and s_- and take the one with the smallest cut as final solution (partition).

Remark: Largest (smallest) s_i^* correspond exactly to the largest (smallest) $(v_2)_i$.

To sum up: The Spectral Partitioning Algorithm (input: A, n_1, n_2)

① Compute the graph Laplacian $L = D - A$, $D = \mathrm{diag}(\underbrace{k_1 \ldots k_n}_{\text{degrees}})$

② Compute the Fiedler vector v_2, the eigenvector of L, corresponding to the second smallest eigenvalue λ_2.

③ Compute two partitions s_+ and s_- as in (a) and (b) with v_2 instead of s^* (we don't really need s^*)

(• If graph is connected $\Rightarrow \lambda_2 > 0$ (spectral gap)
• If not $\Rightarrow \lambda_2 = 0$)

④ Between s_+ and s_- choose the partition that gives the smaller cut size $R = \frac{1}{4} s^T L s$.